

AI Interactions and Critical Evaluation

Appendix to the MFE 230GA final project, Group 4

October 2026

How to read this appendix

The brief asks for at least three substantial AI interactions (idea generation, coding support, robustness design), each documented with the prompt, a summary of the output, and a critical evaluation. Sections 1–3 cover the three interactions behind the main project (analyst revisions vs. industry momentum). They come from the project’s interaction log (`ai_log/ai_interactions.md` in 230GA-Final-Project-charisma), which recorded each prompt verbatim at the time. Section 4 summarizes the AI use in the Climate Alpha stage, and Section 5 reflects on AI as an investment research assistant.

Tools. Claude Code (Anthropic) was the coding and research assistant for the main project. A second AI chat tool was used for one methodology question (Section 1). The brief names ChatGPT; we used ChatGPT in Homework 1 and in the first Climate Alpha implementation, and Claude afterwards.

What the prompts looked like. Most follow-up prompts in the log are short (“ok”, “yes do steps 1-2 then start phase 5”). The real specification was the project plan (`CLAUDE.md`) that we attached to the first prompt and that the assistant re-read before every phase. It covers the prompt components from the discussion session: role and audience (a course project for a grader), task (nine phases with outputs), context (data filters, parameters, sample), output format (tables, figures, tests), and “do not” rules. We quote the relevant parts of it below. Section 5 discusses what the short follow-ups cost us.

Disclosure. The log left the “critical evaluation” sections blank for the team. The evaluations below were written for this submission from the facts recorded in the log, the code history and the rerun results. Each states what was useful, what was wrong or incomplete, what we changed, and the decision that followed.

1. Research design and data review (idea generation / design)

Date and phase. 2 to 4 October 2026; project setup and Phases 2–3 (cleaning, signals).

Prompt (verbatim). First prompt, with the plan attached as a file:

[The project plan `CLAUDE.md` attached as a file, with no other text.] https://github.com/charishma005/Equities_Course_Project I want to set up this repo for the project

Plan excerpts that acted as the specification (verbatim from `CLAUDE.md`):

Thesis (falsifiable, fixed before testing): Earnings news diffuses slowly, so industries with net upward analyst estimate revisions outperform over the following month, and this revision signal carries information beyond price momentum.

Rejection criterion (decided in advance): If the revision signal (or the blended strategy) shows no statistically significant alpha ($|t| < 2$) after controlling for FF5 + UMD, net of costs, and particularly in the post-2010 sample, the report concludes “do not implement.”

A signal formed at the end of month t may only use information available on or before the end of month t . It predicts the return in month $t+1$. [...] Never fit anything on the full sample and then apply it in-sample.

Review prompts in the second session (verbatim):

how is this looking so far; any improvements needed?

what are the next steps now? currently ignoring 2026

[mid-task, pasting advice from another AI tool] For the primary results, don't include 2026 REV or REV_ALT. [...] Keep the planned March 2025–August 2026 review window and report how many months each signal actually covers; don't shift the window to make coverage look complete. [...] so should i include 2026 now or not in reviewing my results?

Output summary. The assistant set up the repository, wrote the WRDS and public-data pull code with offline tests, and later reviewed the team's cleaning and signal code against the plan. The review found two real errors: momentum compounded months t-12 to t-2 instead of the planned t-11 to t-1, and CRSP market caps and SIC codes were carried forward across mid-sample gaps instead of only after CRSP ends. It traced the missing 2026 revisions to the annually updated I/B/E/S–CRSP link table, listed three alternatives (extend active links, CUSIP matching, I/B/E/S prices), added coverage tables, a MOM–REV correlation figure and a test that future data cannot change past signals.

Critical evaluation.

- *Useful.* The code review was the most valuable output of the whole project. The momentum window error would have shifted the signal by a month, and neither error was visible in summary statistics. The future-data invariance test it added is now one of 68 passing tests.
- *Wrong.* When we wrote “currently ignoring 2026”, the assistant added a December 2025 sample cutoff. That silently moved the required “most recent 18 months” to July 2024–December 2025, a window chosen so that coverage looks complete. The brief requires the most recent 12–18 months and warns against reporting only the convenient sample. The well-written output made the change easy to miss; we caught it when the second AI tool's advice, which we pasted into the session, said the opposite.
- *Incomplete.* The second AI tool's advice was correct, but we did not log the prompt we sent it, so that interaction cannot be documented in full. That is a gap in our record.
- *What we changed.* We removed the cutoff, kept REV missing in 2026, kept the March 2025–August 2026 window, and reported how many months each signal covers (MOM 17, REV 10). The rule went into the plan so later phases followed it.
- *Decision.* The recent window is reported as required and labeled low power. This choice later mattered: the blend's only $t \geq 2$ alpha is in that 10-month window, and our rule prevented it from being read as evidence.

2. Coding and implementation support (risk model, sizing, backtest)

Date and phase. 4 October 2026; Phases 4–6.

Prompts (verbatim).

what's next steps after u review the git hub

yes do steps 1-2 then start phase 5 and Still open: the repo is public and contains derived WRDS outputs and real AAPL values. Make it private unless your course says otherwise. - ignore for now we will see in the end

how are the risks looking so far; ok go with covariance shrinkage and go with phase 6;

Plan excerpt for Phase 5 (verbatim): “Covariance of the 49 industry excess returns, exponentially weighted with a **30-month half-life**, estimated each month on data through month t only. [...] Choose λ each month so ex-ante active risk $\sqrt{h' \Sigma h}$ equals the target ($\text{TARGET_ACTIVE_RISK} = 0.05$ annualized). [...] The report must state λ and the active risk it implies.”

Output summary. The assistant reviewed the IC code (no look-ahead found), flagged at the planned checkpoint that REV orthogonal to momentum has an IC of about zero, and wrote the risk model (EWMA covariance, residual volatilities), Grinold–Kahn alphas, a dollar-neutral mean-variance optimizer with a 10%-of-gross position cap, the HW2 diagonal rule as a comparison, monthly λ calibration, and the backtest with drift-adjusted turnover and 10/20/30 bp costs. It reported that the unshrunk books hit 5% ex-ante risk but realized about 10%

with roughly 4x gross, ran a side test showing that shrinking the covariance toward its diagonal closes most of the gap, and left the choice to us.

Critical evaluation.

- *Useful.* The implementation was correct on the first pass in the places that are easy to get wrong: the covariance at month t uses only data through t , the cost is charged on the next month's return, and turnover is measured against drifted weights. Its unit tests check these by hand. It also raised the realized-versus-ex-ante risk gap itself, which a less careful implementation would have hidden behind the "5% ex-ante" number.
- *Wrong.* An earlier session had written type hints such as `pd.Series | pd.DatetimeIndex`, which need Python 3.10+. The assistant tested only in its own Python 3.11 sandbox, so every script crashed on a teammate's Python 3.9 Mac. The model assumed its environment was ours.
- *Incomplete.* The assistant did not ask whether 50% shrinkage was the right amount; it offered it as "closes most of the gap". The choice was made after seeing realized risk, which is a risk-model decision rather than a return-based one, but it is still a design choice informed by the sample, and we say so in the report's limitations.
- *What we changed.* We chose 50% shrinkage and recorded it in the plan (realized volatility now 4.3–6.6% against 5%, gross 1.2–1.5x). All modules got from `__future__ import annotations` and were retested under Python 3.9 and pandas 2.2.3.
- *Decision.* The report states λ (blend median 2.32, 10th–90th percentile 1.93–3.23) and reports realized risk next to the target, including the 1.32x overshoot of the blend, which comes from the 2009 momentum crash.

3. Robustness and factor-attribution design

Date and phase. 4 October 2026; Phases 7–9.

Prompts (verbatim). Each was a one-word approval of the planned phase:

ok

(in reply to "Should I go ahead [with Phase 7]?"; the same for Phase 8 and Phase 9.)

Plan excerpt (verbatim): "Regress monthly **net** returns (20 bps base case) of each strategy on: **Mkt-RF, SMB, HML, RMW, CMA, UMD** (6 factors). Robustness specification: add the Short-Term Reversal factor (7 factors). [...] Key comparison table: alpha of REV and BLENDED with vs. without UMD in the regression. [...] Run for full sample, post-2010, and most recent 18 months (note low power in the short window; report it anyway)." The Phase 8 grid (momentum lookback, revision measure, minimum analysts, half-life, risk target, costs, industry set, neutrality) was also specified in the plan.

Output summary. The assistant wrote the attribution module (FF5, FF5+UMD, FF5+UMD+STREV, Newey–West 6 lags, a check that stops the run if any return month has no factor data), the with/without-UMD table with a rejection-criterion flag, the one-at-a-time robustness grid where each row reruns the whole pipeline, a 3x3 lookback-by-half-life heatmap, annual returns, a 2009/2020 stress table and a rolling 36-month alpha. It also drafted the report summary from the result tables.

Critical evaluation.

- *Useful.* The design is the right one for our question: the with/without-UMD comparison is exactly the test of whether the blend's alpha is new or momentum, and it showed the blend's FF5 alpha of 2.4% ($t = 2.21$) falling to 0.2% ($t = 0.36$). Rerunning the full pipeline for each grid row, rather than reusing holdings, makes the grid an honest test of design choices.
- *Wrong (1).* The `passes_criterion` flag marked the blend's recent window as a pass ($t = 3.40$). That regression has 10 observations for 7 parameters with 6 Newey–West lags; the t -statistic means nothing. The plan said "note low power", but the code had no guard. Had we read the flag instead of the sample size, the table would have shown a "pass".
- *Wrong (2).* The first draft summary then said that no strategy or window reached $t \geq 2$, which was false (the same 10-month window). The assistant caught this on its own review, but it shows that its prose summaries need checking against the tables.

- *Wrong* (3). The row-alignment code in `attribution.align` built a mask before a merge that renumbers rows. Under pandas 2.2 on a teammate’s machine it crashed when given a filtered subset, so the rolling alpha was not written. Its tests only used freshly numbered rows. The bug could only crash, not misalign silently, so committed results were unaffected.
- *What we changed*. Windows under 36 months are labeled “low power” and cannot pass or fail; the regression is still reported. The summary sentence was corrected. The alignment was fixed and a regression test now passes a filtered subset. For this submission we also re-estimated the key regressions independently in the notebook (maximum difference 4×10^{-16}) and added gross and cost-level alphas.
- *Decision*. Every strategy fails the pre-registered test in the 1995–2025 and post-2010 windows, and no robustness row has a 6-factor $t \geq 2$ (highest 1.40). The recommendation is “do not implement.”

4. AI use in the Climate Alpha stage (before the pivot)

Homework 1, Question 6 (ChatGPT). We asked ChatGPT for two strategies grounded in our covariance, volatility and beta results. It proposed a factor-neutral emissions spread (Strategy A) and a regime-switched Brown/Green trade (Strategy B). Our evaluation (in the HW1 submission) accepted A, rejected B as a timing rule fitted to three regime turns in 52 years, and noted that ChatGPT flagged a look-ahead problem we had missed: classifying industries as clean in 1975 using a 2020 emissions snapshot. It also wrote down the risk that a hedged spread might be “a growth-minus-value bet with a commodity short attached,” which is what we later found.

First implementation (230GA-Final-Project-main). ChatGPT produced a replication blueprint from the notebook and data files (parameters fixed before coding), and Claude Code implemented the attention-timed strategy and its tests. The team write-up records one serious AI error that was caught: macro R^2 for the raw and purified signals was compared on different sample windows, which flattered the purification. That repository’s `ai/` folder has a template but no transcripts, so these interactions cannot be quoted verbatim.

Research extension (230GA-Final-Project-hashim-research-extension). This audit used four structured exchanges (design review, code for a frozen test, a multiple-testing framework, and a red team of the conclusion), each fact-checked with code. Provenance matters here: the extension was run with AI agents under Hashim’s direction. An agent drafted the prompts in his voice, a separate Claude instance with no files or tools answered them, and fact-check code verified every claim (see that repository’s `REVIEW_NOTES.md`). The most instructive findings were: the replies never asked what the “attention” series measured and accepted its label (it is a stock-market-volatility news index); a look-ahead test written by the model caught only 4 of 6 timing bugs that were deliberately injected; and the model’s code reproduced a frozen test exactly when the specification was exact. We treat these exchanges as supporting evidence, not as the main project’s three interactions.

5. Reflection: strengths and limits of AI as a research assistant

Strengths. Given an exact specification, the assistant was a fast and careful implementer, and a useful second reader of our own code: it found the momentum-window error and the carry-forward error, and it raised the risk-target overshoot before we did. It never fabricated a number in the result tables; every number came from code we could rerun.

Limits. Its errors were errors of judgment and context, not arithmetic. It filled gaps in the plan with plausible defaults that leaned toward cleaner-looking results (a sample cutoff that made coverage look complete, a “pass” flag on a 10-month regression, a summary sentence that overstated the evidence). It assumed its own software environment, and it accepted inputs on trust (in the Climate stage, a mislabeled signal). It also does not supply the economics: it reported that the REV weight in the blend fell from about 0.34 to below zero and concluded the blend was “essentially momentum”, but it offered no explanation and did not suggest a check, such as the subperiod IC split we added, even though that decline was the most informative fact in the study.

What we would do differently. Our one-word follow-ups delegated too much. They worked because the plan fixed the thesis, windows and rejection rule before any result existed; where the plan was silent, the model’s defaults were wrong. A better workflow is to state, in each follow-up prompt, the specific risk we want checked (“does any choice in this phase depend on the result?”), to require the model to list its own assumptions, and

to log every prompt, including those sent to other tools.